

# MODEL 980A COMPUTER ASSEMBLY LANGUAGE CODING CONVENTIONS

MANUAL NO. 961961-9732

REISSUED 15 FEBRUARY 1974

8 JULY 1974

CHANGE 1

NATIONAL BIOMEDICAL RESEARCH FOUNDATION  
Georgetown University Medical Center  
3900 Reservoir Road, N. W.  
Washington, D. C. 20007

NOV 05 1974



**TEXAS INSTRUMENTS**  
INCORPORATED

© Texas Instruments Incorporated 1974  
All Rights Reserved

The information and/or drawings set forth in this document and all rights in and to inventions disclosed herein and patents which might be granted thereon disclosing or employing the materials, methods, techniques or apparatus described herein are the exclusive property of Texas Instruments Incorporated.

No disclosure of the information or drawings shall be made to any other person or organization without the prior consent of Texas Instruments Incorporated.

## LIST OF EFFECTIVE PAGES

INSERT LATEST CHANGED PAGES DESTROY SUPERSEDED PAGES

Note: The portion of the text affected by the changes is indicated by a vertical bar in the outer margins of the page.

Model 980A Computer Assembly Language Coding  
Conventions (961961-9732)

Original Issue . . . . . 2 February 1973  
Reissued . . . . . 15 February 1974 (ECN 365758)  
Change 1 . . . . . 8 July 1974 (ECN 397689)

Total number of pages in this publication is 44 consisting of the following:

| PAGE<br>NO.             | CHANGE<br>NO. | PAGE<br>NO. | CHANGE<br>NO. | PAGE<br>NO. | CHANGE<br>NO. |
|-------------------------|---------------|-------------|---------------|-------------|---------------|
| Title . . . . .         | 1             |             |               |             |               |
| ii . . . . .            | 1             |             |               |             |               |
| iii - v . . . . .       | 0             |             |               |             |               |
| vi Blank . . . . .      | 0             |             |               |             |               |
| 1-1 . . . . .           | 0             |             |               |             |               |
| 1-2 Blank . . . . .     | 0             |             |               |             |               |
| 2-1 . . . . .           | 1             |             |               |             |               |
| 2-2 - 2-6 . . . . .     | 0             |             |               |             |               |
| 2-7 . . . . .           | 1             |             |               |             |               |
| 2-8 - 2-27 . . . . .    | 0             |             |               |             |               |
| 2-28 . . . . .          | 1             |             |               |             |               |
| A-1 - A-2 . . . . .     | 0             |             |               |             |               |
| E-1 . . . . .           | 0             |             |               |             |               |
| B-2 Blank . . . . .     | 0             |             |               |             |               |
| User's Resp . . . . .   | 1             |             |               |             |               |
| Bus. Reply . . . . .    | 1             |             |               |             |               |
| Cover Blank . . . . .   | 0             |             |               |             |               |
| Cover Imprint . . . . . | 0             |             |               |             |               |





## MANUAL UPDATE

---

MANUAL: Model 980A Computer Assembly Language Coding Conventions (961961-9732)

ECN NO.: 397689 (Change 1)

DATE: 8 July 1974

This change package reflects compatability revisions to the manual. The attached pages should be inserted in place of the obsolete pages:

| <u>Obsolete Pages</u> | <u>Updated Pages</u> |
|-----------------------|----------------------|
| Title Page            | Title Page           |
| ii                    | ii                   |
| 2-1/2-2               | 2-1/2-2              |
| 2-7/2-8               | 2-7/2-8              |
| 2-27/2-28             | 2-27/2-28            |
| Comment Sheet         | User's Response      |
| Business Reply        | Business Reply       |
| Back Cover            | Back Cover           |





## COMMENT SHEET

### ASSEMBLY LANGUAGE CODING CONVENTIONS FOR THE MODEL 980A COMPUTER

FROM

NAME \_\_\_\_\_ OFFICE/DEPT. NO. \_\_\_\_\_

CITY/STATE \_\_\_\_\_ DATE \_\_\_\_\_

To make this manual more useful to you, we want your comments: what additional information should be included in the manual; what description or figure could be clarified; what subject requires more explanation; what presentation is particularly helpful to you; and so forth. Thank you.

CUT ALONG LINE

NO POSTAGE NECESSARY IF MAILED IN U. S. A.

FOLD ON TWO LINES (LOCATED ON REVERSE SIDE), STAPLE AND MAIL

FIRST CLASS  
Permit No 6189  
Houston, Texas

**BUSINESS REPLY MAIL**

No postage necessary if mailed in the United States

POSTAGE WILL BE PAID BY

**TEXAS INSTRUMENTS INCORPORATED**

**DIGITAL SYSTEMS DIVISION**

**P.O. BOX 1444 HOUSTON, TEXAS 77001**

**ATTENTION: TECHNICAL PUBLICATIONS, MS 770  
MANUAL CORRECTION**

## TABLE OF CONTENTS

| Section   | Page                                                        |
|-----------|-------------------------------------------------------------|
| <b>I</b>  | <b>GENERAL INFORMATION</b>                                  |
| 1-1       | Scope of Manual . . . . . 1-1                               |
| 1-2       | Related Material . . . . . 1-1                              |
| <b>II</b> | <b>PROGRAM INSTRUCTION CONVENTIONS</b>                      |
| 2-1       | Symbolic Assembly Program (SAP) . . . . . 2-1               |
| 2-2       | Symbolic Line Format . . . . . 2-1                          |
| 2-2.1     | Comment Lines . . . . . 2-6                                 |
| 2-2.2     | Name Field . . . . . 2-6                                    |
| 2-2.3     | Operation Field . . . . . 2-6                               |
| 2-2.4     | Operand Field . . . . . 2-6                                 |
| 2-2.5     | Comment Field . . . . . 2-7                                 |
| 2-3       | Machine Instructions General Symbolic Formats . . . . . 2-8 |
| 2-3.1     | Register-Memory Instructions . . . . . 2-8                  |
| 2-3.2     | Register Shift Instructions . . . . . 2-10                  |
| 2-3.3     | Register-Register Instructions . . . . . 2-10               |
| 2-3.4     | Register Skip Instructions . . . . . 2-11                   |
| 2-3.5     | Status Indicator Skip Instructions . . . . . 2-11           |
| 2-3.6     | Sense Switch Skip Instructions . . . . . 2-11               |
| 2-3.7     | Multi-Register Instructions . . . . . 2-12                  |
| 2-3.8     | Byte Manipulation Instructions . . . . . 2-12               |
| 2-3.9     | Memory Bit Manipulation Instructions . . . . . 2-13         |
| 2-3.10    | Register Bit Manipulation Instructions . . . . . 2-13       |
| 2-3.11    | API Instructions . . . . . 2-14                             |
| 2-3.12    | Data Bus Input/Output Instructions . . . . . 2-14           |
| 2-3.13    | DMAC Input/Output Instructions . . . . . 2-15               |
| 2-3.14    | Single Function Instructions . . . . . 2-15                 |
| 2-4       | Assembler Directives . . . . . 2-16                         |
| 2-4.1     | BES-Block Ending Symbol . . . . . 2-16                      |
| 2-4.2     | BRR-Base Register Reset . . . . . 2-16                      |
| 2-4.3     | BRS-Base Register Set . . . . . 2-16                        |
| 2-4.4     | BSS-Block Starting Symbol . . . . . 2-17                    |
| 2-4.5     | BYTE-Generate Byte Address . . . . . 2-18                   |
| 2-4.6     | COMM-Blank Common . . . . . 2-20                            |
| 2-4.7     | DATA-Generate Word Address or Data . . . . . 2-20           |
| 2-4.8     | DEF-Define Entry Point Symbol . . . . . 2-21                |
| 2-4.9     | END-End of Source . . . . . 2-22                            |
| 2-4.10    | EQU-Equate . . . . . 2-23                                   |
| 2-4.11    | FLAG-Flag Bit Address . . . . . 2-23                        |
| 2-4.12    | FRM-Format A New Instruction . . . . . 2-23                 |



## TABLE OF CONTENTS (Continued)

| Section                                          | Page |
|--------------------------------------------------|------|
| 2-4.13 HED-Page Heading . . . . .                | 2-24 |
| 2-4.14 IDT-Object Identifier . . . . .           | 2-24 |
| 2-4.15 IF-Conditional Assembly . . . . .         | 2-24 |
| 2-4.16 LIS-Start Listing . . . . .               | 2-25 |
| 2-4.17 OPD-Operation Define . . . . .            | 2-25 |
| 2-4.18 ORG-Origin . . . . .                      | 2-26 |
| 2-4.19 PEJ-Page Eject . . . . .                  | 2-27 |
| 2-4.20 REF-Referenced External Symbols . . . . . | 2-27 |
| 2-4.21 UNL-Stop Listing . . . . .                | 2-27 |
| 2-5 Object Format . . . . .                      | 2-27 |
| 2-6 SAP Error Messages . . . . .                 | 2-28 |
|                                                  |      |
| APPENDIX A                                       |      |
| SYMBOLIC FORM OF MACHINE INSTRUCTIONS . . . . .  | A-1  |
| DATA GENERATION . . . . .                        | A-2  |
| ASSEMBLER DIRECTIVES . . . . .                   | A-2  |
|                                                  |      |
| APPENDIX B                                       |      |
| IXB ADDRESSING EXAMPLES . . . . .                | B-1  |

## LIST OF ILLUSTRATIONS

| Figure |                                  | Page |
|--------|----------------------------------|------|
| 2-1    | Coded Main Program . . . . .     | 2-2  |
| 2-2    | Coded Subroutine . . . . .       | 2-3  |
| 2-3    | Assembled Main Program . . . . . | 2-4  |
| 2-4    | Assembled Subroutine . . . . .   | 2-5  |
| 2-5    | END-OF-LINE Characters . . . . . | 2-6  |
| 2-6    | Example of BYTE Usage . . . . .  | 2-19 |
| 2-7    | Example of DATA Usage . . . . .  | 2-21 |

## LIST OF TABLES

| Table |                                                          | Page |
|-------|----------------------------------------------------------|------|
| 1-1   | Related Manuals . . . . .                                | 1-1  |
| 2-1   | Register-Memory Format Symbolic Interpretation . . . . . | 2-9  |



## SECTION I

### GENERAL INFORMATION

#### 1-1 SCOPE OF MANUAL.

This is the second in a series of three manuals describing the Model 980A assembly language. Section II of this manual covers the Symbolic Assembly Program (SAP), the assembly form of machine instructions, and the available assembler directives. A summary of the symbolic form of machine instructions and assembler directives is given in Appendix A. IXB addressing examples are given in Appendix B.

#### 1-2 RELATED MATERIAL.

The first manual in this series is Assembly Language Machine Instructions for the Model 980A Computer. If stand alone programs are to be prepared, the third manual in the series should be consulted: Assembly Language Input/Output for the Model 980A Computer. Optionally, the Basic System may be used to perform all input/output, in which case the needed information may be found in the manual, Basic System Use and Operation for the Model 980A Computer. The same manual contains the information necessary to load and execute a machine language program. or to assemble, load, and execute a stand alone assembly language program.

A list of all software and hardware manuals available may be found in the Software Manual Index for the Model 980A Computer. The manual numbers listed in Table 1-1 must be included when ordering 980A software manuals.

TABLE 1-1. RELATED MANUALS

| MANUAL                                                             | MANUAL NO.  |
|--------------------------------------------------------------------|-------------|
| Assembly Language Machine Instructions for the Model 980A Computer | 961961-9730 |
| Assembly Language Input/Output for the Model 980A Computer         | 961961-9734 |
| Basic System Use and Operation for the Model 980A Computer         | 961961-9710 |
| Software Manual Index for the Model 980A Computer                  | 961961-9792 |





## SECTION II

### PROGRAM INSTRUCTION CONVENTIONS

#### 2-1 SYMBOLIC ASSEMBLY PROGRAM (SAP).

Several versions of SAP are available to translate symbolic coding into object language acceptable to the Model 980A Computer. Figure 2-1 is a sample source main program, written in symbolic assembly language, ready to be punched into cards or paper tape or otherwise prepared for input. Figure 2-2 is a source subroutine. A source program is input to SAP which generates two outputs. The first is an object program that can be loaded into the computer and executed or linked with other object programs. The object program can be output on punched cards, paper tape, or other media. The second output is an assembly listing as depicted in Figure 2-3 for the main program and Figure 2-4 for the subroutine. Note the following in the output examples:

- a. The items listed under A are an exact reproduction of the handwritten entries on the coding sheet.
- b. The items under B are a hexadecimal representation of the corresponding instructions and constants.
- c. The items under C show the hexadecimal addresses of the instructions, constants, and areas of storage specified by the programmer.
- d. The items under D show the decimal line or sequence number of the source statements to be used in case the program is changed.

SAP is a two-pass assembler, that is, it scans the source program twice. During the first pass, the source program is read and a symbol table is generated. This is accomplished through the use of a location counter in the assembler. The location counter keeps track of the storage locations that will be required by the object program. When a source statement contains a name, the current setting of the location counter is assigned to the name. Each name and the address assigned to it is placed in the assembler's symbol table. During the second pass, the symbol table is used to complete the assembly, and to produce the object with its assembly listing. If bulk storage is available, some versions of SAP copy the source to bulk storage during pass one. Since the output from the first pass is used as input data for the second pass, this eliminates the requirement to manually enter the source data twice.

#### 2-2 SYMBOLIC LINE FORMAT.

The symbolic input line accepted by the assembler may contain a name field, operation field, operand field, and a comment field – or the entire line may be a comment. An input line is the first 64 characters read from a card or in the case of paper tape, an input line is a string of characters the last of which are shown in Figure 2-5. The input line must not exceed 64 characters, not including the end-of-line characters (paper tape only). Input lines are free form within the limits described herein. (Only 59 characters are output to LUN 6.) (Continued on page 2-6)

## SYMBOLIC CODING FORM

|        | 5    | 10     | 15           | 20         | 25       | 30       | 35 | 40 | 45 |
|--------|------|--------|--------------|------------|----------|----------|----|----|----|
|        | HED  | MODEL  | 980A         | MAIN       | PROGRAM  |          |    |    |    |
|        | IDT  | ILLUS  | 6            | CHARS.     | FOR      | OBJECT   |    |    |    |
|        | ORG  | 1000   | TELL         | SAP        | RUN-TIME |          |    |    |    |
|        | BRS  | 1000   | ORIGIN       | AND        | BASE     |          |    |    |    |
| BASE   | DATA | 1000   |              |            |          |          |    |    |    |
|        | REF  | SUB    | EXT.         | REF.       | FOR      | LINKING  |    |    |    |
| START  | LDA  | BASE   | ACTUALLY     | SET        | BASE     |          |    |    |    |
|        | RMO  | 0,6    | FOR          | EXECUTION  |          |          |    |    |    |
|        | @BRL | SUB    | ADD          | 2          | NOS.     | TOGETHER |    |    |    |
|        | DATA | ADDR1  | ADDR.OF      | FIRST      | NO.      |          |    |    |    |
|        | DATA | ADDR2  | ADDR.OF      | SECOND     | NO.      |          |    |    |    |
|        | STA  | ANSWER | ANSWER       | IN         | REG.A    |          |    |    |    |
| .      | .    |        | MORE         | EXECUTABLE |          |          |    |    |    |
| .      | .    |        | INSTRUCTIONS | AND        |          |          |    |    |    |
| .      | .    |        | ASSEMBLER    |            |          |          |    |    |    |
| .      | .    |        | DIRECTIVES.  |            |          |          |    |    |    |
| ADDR1  | DATA | 7      | FIRST        | NO.        |          |          |    |    |    |
| ADDR2  | DATA | 8      | SECOND       | NO.        |          |          |    |    |    |
| ANSWER | BSS  | 1      |              |            |          |          |    |    |    |
|        | END  | START  |              |            |          |          |    |    |    |

FIGURE 2-1. CODED MAIN PROGRAM

# SYMBOLIC CODING FORM

|       | 5 | 10 | 15  | 20       | 25 | 30 | 35 | 40 | 45                      |
|-------|---|----|-----|----------|----|----|----|----|-------------------------|
|       |   |    | IDT | SUB      |    |    |    |    | 6 CHARS.FOR OBJECT      |
|       |   |    | DEF | SUB      |    |    |    |    | DEFINE ENTRY POINT FOR  |
| A     |   |    | EQU | 0        |    |    |    |    | LINKING                 |
| L     |   |    | EQU | 5        |    |    |    |    | GIVE REGISTERS SYMBOLIC |
| P     |   |    | EQU | 7        |    |    |    |    | NAMES.                  |
| POINT |   |    | BSS | 2        |    |    |    |    | RESERVE LOCATIONS.      |
| HERE  |   |    | BSS | 2        |    |    |    |    |                         |
| SUB   |   |    | RMO | L,A      |    |    |    |    | L POINTS TO FIRST DATA  |
|       |   |    | STA | POINT    |    |    |    |    | WORD AFTER @BRL         |
|       |   |    | RIN | A,A      |    |    |    |    | POINTER TO SECOND DATA  |
|       |   |    | STA | POINT+1  |    |    |    |    | WORD AFTER @BRL         |
|       |   |    | LDA | *POINT   |    |    |    |    | GET ADDR1               |
|       |   |    | STA | HERE     |    |    |    |    | STORE ADDRESS IN THIS   |
|       |   |    | LDA | *POINT+1 |    |    |    |    | SUBROUTINE              |
|       |   |    | STA | HERE+1   |    |    |    |    | GET AND SAVE ADDR2      |
|       |   |    | LDA | *HERE    |    |    |    |    | PICK UP FIRST NO.       |
|       |   |    | ADD | *HERE+1  |    |    |    |    | ADD SECOND NO.          |
|       |   |    | RIN | L,L      |    |    |    |    | MOVE POINTER PAST DATA  |
|       |   |    | RIN | L,P      |    |    |    |    | WORDS AND RETURN.       |
|       |   |    | END | SUB      |    |    |    |    |                         |

FIGURE 2-2. CODED SUBROUTINE

|      |      |      |       |                         |                     |
|------|------|------|-------|-------------------------|---------------------|
|      |      | 0001 | HED   | MODEL 980A MAIN PROGRAM |                     |
|      |      | 0002 | IDT   | ILLUS                   | 6 CHARS.FOR OBJECT  |
| 03E8 |      | 0003 | ORG   | 1000                    | TELL SAP RUN-TIME   |
|      | 03E8 | 0004 | BRS   | 1000                    | ORIGIN AND BASE     |
| 03E8 | 03E8 | 0005 | BASE  | DATA                    | 1000                |
|      |      | 0006 | REF   | SUB                     | EXT.REF.FOR LINKING |
| 03E9 | 00FE | 0007 | START | LDA                     | BASE                |
| 03EA | C506 | 0008 |       | RMO                     | 0,6                 |
| 03EB | 7400 | 0009 |       | @BRL                    | SUB                 |
|      |      |      |       |                         | ADD 2 NOS. TOGETHER |
|      |      | 0000 |       |                         |                     |
| X    | 03EC | 0000 |       |                         |                     |
|      | 03ED | 03F0 | 0010  | DATA                    | ADDR1               |
|      | 03EE | 03F1 | 0011  | DATA                    | ADDR2               |
|      | 03EF | 8002 | 0012  | STA                     | ANSWER              |
|      |      |      |       |                         | ADDR.OF FIRST NO.   |
|      |      |      |       |                         | ADDR.OF SECOND NO.  |
|      |      |      |       |                         | ANSWER IN REG.A     |
|      |      |      |       |                         | MORE EXECUTABLE     |
|      |      |      |       |                         | INSTRUCTIONS AND    |
|      |      |      |       |                         | ASSEMBLER           |
|      |      |      |       |                         | DIRECTIVES.         |
|      | 03F0 | 0007 | 0017  | ADDR1                   | DATA                |
|      | 03F1 | 0008 | 0018  | ADDR2                   | DATA                |
|      | 03F2 |      | 0019  | ANSWER                  | BSS                 |
|      |      |      | 0020  | REF                     | SUB1                |
| X    | 03F3 | 0000 | 0021  | SUB2                    | DATA                |
|      |      | 0000 |       |                         | SUB1                |
|      |      | 0000 | 0022  | WORD                    | COMM                |
|      |      | 0000 | 0023  | DATA                    | WORD+2              |
| C    | 03F4 | 0002 |       |                         |                     |
|      |      | 0004 | 0024  | END                     | START               |

|                 |   |             |      |        |      |        |      |      |      |
|-----------------|---|-------------|------|--------|------|--------|------|------|------|
| Symbol<br>Table | { | ADDR1       | 03F0 | SUB    | 0000 | START  | 03E9 | BASE | 03E8 |
|                 |   | ADDR2       | 03F1 | ANSWER | 03F2 | R SUB2 | 03F3 | WORD | 03F3 |
|                 |   | 0000 ERRORS |      |        |      |        |      |      |      |

NOTES:

1. The symbol table is not generated by the 4K version of the assembler.
2. In the left column, P = Program counter relocatable  
     X = External reference  
     C = Common (to programs)
3. In the symbol table, R = Unreferenced symbol  
     U = Undefined (error)  
     M = Multidefined
4. A, B, C, & D references at top of page are explained on page 2-1.

FIGURE 2-3. ASSEMBLED MAIN PROGRAM

|        |      |            | A   |          |                         |
|--------|------|------------|-----|----------|-------------------------|
| C      | B    | D          |     |          |                         |
|        |      | 0001       | IDT | SUB      | 6 CHARS.FOR OBJECT      |
|        |      | 0002       | DEF | SUB      | DEFINE ENTRY POINT FOR  |
|        | 0000 | 0003 A     | EQU | 0        | LINKING                 |
|        | 0005 | 0004 L     | EQU | 5        | GIVE REGISTERS SYMBOLIC |
|        | 0007 | 0005 P     | EQU | 7        | NAMES.                  |
|        |      | 0006 POINT | BSS | 2        | RESERVE LOCATIONS.      |
| P 0000 |      | 0007 HERE  | BSS | 2        |                         |
| P 0002 |      | SUB        | RMO | L,A      | L POINTS TO FIRST DATA  |
| 0004   | C550 | 0008       | STA | POINT    | WORD AFTER @BRL         |
| 0005   | 80FA | 0009       | RIN | A,A      | POINTER TO SECOND DATA  |
| 0006   | C300 | 0010       | STA | POINT+1  | WORD AFTER @BRL         |
| 0007   | 80F9 | 0011       | LDA | *POINT   | GET ADDR1               |
| 0008   | 04F7 | 0012       | STA | HERE     | STORE ADDRESS IN THIS   |
| 0009   | 80F8 | 0013       | LDA | *POINT+1 | SUBROUTINE              |
| 000A   | 04F6 | 0014       | STA | HERE+1   | GET AND SAVE ADDR2      |
| 000B   | 80F7 | 0015       | LDA | *HERE    | PICK UP FIRST NO.       |
| 000C   | 04F5 | 0016       | ADD | *HERE+1  | ADD SECOND NO.          |
| 000D   | 24F5 | 0017       | RIN | L,L      | MOVE POINTER PAST DATA  |
| 000E   | C355 | 0018       | RIN | L,P      | WORDS AND RETURN.       |
| 000F   | C357 | 0019       | END | SUB      |                         |
|        | 0004 | 0020       |     |          |                         |

|              |       |        |      |      |   |      |   |      |
|--------------|-------|--------|------|------|---|------|---|------|
| Symbol Table | A     | 0000   | HERE | 0002 | L | 0005 | P | 0007 |
|              | POINT | 0000   | SUB  | 0004 |   |      |   |      |
|              | 0000  | ERRORS |      |      |   |      |   |      |

## NOTES:

1. The symbol table is not generated by the 4K version of the assembler.
2. A, B, C, & D references at top of page are explained on page 2-1.
3. Symbols in the far left column and in the symbol table are explained in Figure 2-3.

FIGURE 2-4. ASSEMBLED SUBROUTINE



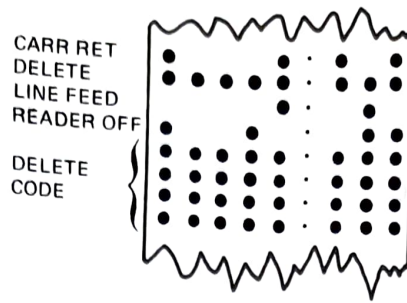


FIGURE 2-5. END-OF-LINE CHARACTERS

In typing to the ASR cassette, the input line is variable length. The final characters for one source line should be in the following order:

|         |                                                 |
|---------|-------------------------------------------------|
| CR      | Carriage Return                                 |
| LF      | Line Feed                                       |
| X-OFF   | (Press the CTRL key and S key at the same time) |
| RUB OUT |                                                 |

**2-2.1 COMMENT LINES.** Comment lines provide the user with the ability to annotate program listings. They are indicated by an initial character which is either a period (.) or an asterisk (\*). The remaining characters are arbitrary. The comment line in no way affects the assembly process. The line is merely reproduced in the printed output.

**2-2.2 NAME FIELD.** Names (also called symbols or labels) are provided for symbolic references to instructions, values, and data. A label is composed of from one to six characters. The first character of label must be a letter. The remaining may be any characters except the following:

|   |       |   |          |   |              |   |              |
|---|-------|---|----------|---|--------------|---|--------------|
| + | Plus  | * | Asterisk | ( | Left Paren.  | > | Greater Than |
| - | Minus | / | Slash    | ) | Right Paren. | , | Comma        |

If a label is used, the first character must begin the input line. The label is terminated by the first space.

At assembly time, the labels are stored as variable length data. One or two character labels require one word of memory, three or four character labels take two words, and five or six characters require three words. Therefore, if the symbol overflow error occurs, labels should be shortened or omitted.

**2-2.3 OPERATION FIELD.** The operation field describes the required action. It may be a mnemonic operation code or an assembler directive. The field consists of from one to four characters followed by a space or the end-of-line characters. The first character of the operation field must be preceded by at least one space.

**2-2.4 OPERAND FIELD.** The operand field consists of a sequence of *expressions* separated by commas and is terminated by a space or the end-of-line characters.

*exp1, exp2, exp3*

If two commas appear successively, the value of the missing expression is understood to be zero. If fewer than the required number of expressions appear in a source line, the missing expressions are assumed to be zero. If the currency symbol (\$) appears as an element in an expression, the current value of the assembler's location counter is used as its numeric equivalent.

Expressions are strings of items separated by arithmetic operators and terminated by a space, comma, or end-of-line characters. The arithmetic operators are:

|                |   |
|----------------|---|
| Addition       | + |
| Subtraction    | - |
| Multiplication | * |
| Division       | / |

If two operators appear in succession, a zero item is assumed.

An item consists of a symbolic address, dollar sign (\$), or a numeric value. If the first character of an item is not numeric, \$, or >, it is assumed to be symbolic. Numeric items may be octal, decimal, or hexadecimal. An octal item is a string of octal characters (0-7), the first of which is zero. A decimal item is a string of numeric characters (0-9), the first of which is non-zero. A hexadecimal item is a greater than symbol (>) followed by a string of hexadecimal digits (0-9, A-F). When using paper tape input, the back slash (\) may be used in place of > to indicate hexadecimal.

Expressions are evaluated left to right using normal arithmetic precedence; i.e., all multiplications and divisions are performed in order of occurrence. The additions and subtractions are then performed in order of occurrence.

All quantities are treated as integers. In division only the quotient is retained and any remainder is discarded. Division by zero is performed as division by one and is not considered as an error.

Sample expressions are:

```
JOE+TOM*3/BOB
$+5
LEA-6
5034
XYZ>F4
```

#### NOTE

All expressions are acceptable in absolute assemblies, but multiplication and division involving labels is not allowed in relocatable assemblies. Hence, the first sample would cause a relocation error in a relocatable program.

**2.2.5 COMMENT FIELD.** Comments may optionally be written on any line. Any characters which appear between the space which terminates the operand field and the end-of-line characters or card column 64 are treated as commentary. The comment field has no effect on the assembly process.

## 2-3 MACHINE INSTRUCTION GENERAL SYMBOLIC FORMATS.

2-3.1 REGISTER-MEMORY INSTRUCTIONS. The general symbolic format of a register-memory instruction is:

| NAME  | OPERATION | OPERAND     |
|-------|-----------|-------------|
| LABEL | @OP       | =*DISP,MODE |

The name field, the asterisk (\*) which denotes indirect addressing, the at (@) symbol which denotes extended format, and the equal (=) which denotes immediate operand, are optional. Both the = and \* may not be present. The MODE item indicates the addressing mode and may have a value of 0 through 7 (bottom of Table 2-1). It is the value of the IXB bits of the object instruction. DISP and MODE are symbolic expressions. Typical symbolic statements are:

| NAME | OPERATION | OPERAND                         |
|------|-----------|---------------------------------|
| X    | EQU       | 2                               |
|      | :         |                                 |
|      | :         |                                 |
| PT1  | LDA       | BUFFER+2                        |
|      | STA       | *ADDR                           |
| PT2  | LDX       | =-5 (-5,7 ALTERNATE OPERAND)    |
|      | @STA      | ANS,X (ANS,2 ALTERNATE OPERAND) |
|      | BIX       | \$-2                            |

### NOTE

Instructions DLD, DST, DAD, DSB, with the IXB and displacement field zero cause the program counter to be incremented twice and the two words immediately following the instruction to be treated as data. However, the assembler generates only one word of data following the instruction when using the symbols @DLD, @DST, etc., as with the other extended format register-memory instructions.

Table 2-1 defines the transliteration process performed by the assembler for register-memory instructions. Table 2-1 uses symbolic indicators for the mode expressions. In practice, the user must write EQU (equate) statements for the assembler to interpret any such symbolism, such as that shown for the mode numbers 0 through 7 at the bottom of Table 2-1. Note that it may be preferred to have the label B equated to register number 6; thus, a different label will have to be used for mode B = 1.

In order to translate the operand address expression in register-memory format instructions, the assembler first evaluates the expression as a 16-bit number and then performs one of the following operations:

- A number one greater than the location counter is subtracted to make the address program counter relative.
- A number given by a BRS directive is subtracted to make the address base register relative.

TABLE 2-1. REGISTER-MEMORY FORMAT SYMBOLIC INTERPRETATION

| Mode Indicator Used                                                                |     |     | (IXB)<br>Mode<br>Expression | Instruction Is In Range        |                              | Base Register<br>Assumed Value<br>Given by Pseudo-Op | Assembler Action                        |
|------------------------------------------------------------------------------------|-----|-----|-----------------------------|--------------------------------|------------------------------|------------------------------------------------------|-----------------------------------------|
| Symbol                                                                             |     |     |                             | Program<br>Counter<br>Relative | Base<br>Register<br>Relative |                                                      |                                         |
| (a)                                                                                | =   | *   |                             |                                |                              |                                                      |                                         |
| NO                                                                                 | NO  | NO  | NONE                        | YES                            | N/A                          | N/A                                                  | IXB=0, D=P relative                     |
| NO                                                                                 | NO  | NO  | X                           | YES                            | N/A                          | N/A                                                  | IXB=2, D=P relative                     |
| NO                                                                                 | NO  | NO  | I                           | YES                            | N/A                          | N/A                                                  | IXB=4, D=P relative                     |
| NO                                                                                 | NO  | YES | NONE or I                   | YES                            | N/A                          | N/A                                                  | IXB=4, D=P relative                     |
| NO                                                                                 | NO  | NO  | NONE                        | NO                             | YES                          | YES                                                  | IXB=1, D=B relative                     |
| NO                                                                                 | NO  | NO  | X                           | NO                             | YES                          | YES                                                  | IXB=3, D=B relative                     |
| NO                                                                                 | NO  | NO  | I                           | NO                             | YES                          | YES                                                  | IXB=5, D=B relative                     |
| NO                                                                                 | NO  | YES | NONE or I                   | NO                             | YES                          | YES                                                  | IXB=5, D=B relative                     |
| NO                                                                                 | NO  | NO  | B                           | N/A                            | N/A                          | NO                                                   | IXB=1, D = absolute                     |
| NO                                                                                 | NO  | NO  | XB                          | N/A                            | N/A                          | NO                                                   | IXB=3, D = absolute                     |
| NO                                                                                 | NO  | NO  | IB                          | N/A                            | N/A                          | NO                                                   | IXB=5, D = absolute                     |
| NO                                                                                 | NO  | YES | B or IB                     | N/A                            | N/A                          | NO                                                   | IXB=5, D = absolute                     |
| NO                                                                                 | NO  | NO  | B                           | N/A                            | YES                          | YES                                                  | IXB=1, D=B relative                     |
| NO                                                                                 | NO  | NO  | XB                          | N/A                            | YES                          | YES                                                  | IXB=3, D=B relative                     |
| NO                                                                                 | NO  | NO  | IB                          | N/A                            | YES                          | YES                                                  | IXB=5, D=B relative                     |
| NO                                                                                 | NO  | YES | B or IB                     | N/A                            | YES                          | YES                                                  | IXB=5, D=B relative                     |
| NO                                                                                 | NO  | NO  | IX                          | YES                            | N/A                          | N/A                                                  | IXB=6, D=P relative                     |
| NO                                                                                 | NO  | YES | X or IX                     | YES                            | N/A                          | N/A                                                  | IXB=6, D=P relative                     |
| NO                                                                                 | NO  | NO  | M                           | N/A                            | N/A                          | N/A                                                  | IXB=7, D = absolute                     |
| NO                                                                                 | NO  | YES | XB or M                     | N/A                            | N/A                          | N/A                                                  | IXB=7, D = absolute                     |
| NO                                                                                 | YES | N/A | N/A                         | N/A                            | N/A                          | N/A                                                  | IXB=7, D = absolute                     |
| YES                                                                                | YES | N/A | NONE                        | N/A                            | N/A                          | N/A                                                  | IXB=0, D=0, 2nd word = immediate value  |
| YES                                                                                | NO  | N/A | NONE                        | N/A                            | N/A                          | N/A                                                  | IXB=4, D=0, 2nd word = indirect address |
| YES                                                                                | NO  | N/A | X                           | N/A                            | N/A                          | N/A                                                  | IXB=6, D=0, 2nd word = indirect address |
| Values given symbolically in the MODE expression column have the following values: |     |     |                             | NONE = 0<br>B = 1              | X = 2<br>XB = 3              | I = 4<br>IB = 5                                      | IX = 6<br>M = 7                         |



- c. The number is used as is in extended format.
- d. The number is truncated to an 8-bit value. This is done for single length immediate value references, and for base relative addressing under a base register reset (BRR) condition.
- e. If the resulting address is unattainable under the defined conditions, a field size error is indicated.

ALA 2-3.2 REGISTER SHIFT INSTRUCTIONS. Shift instructions have the symbolic format:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | OP        | COUNT   |

CRD The label field is optional. COUNT is an expression which is evaluated and used for the shift count.

CRS Typical examples are:

|     |     |        |
|-----|-----|--------|
| CRX | ALD | 1      |
| LLA | ALD | MPYBY2 |
| LLD | ARA | 2      |
| LRA | ARA | DIVBY4 |
| LRD | LRA | 8      |
| LTO | CRE | 8      |
| LTZ | LLD | 8      |
| RTO | LTO | 3      |
| RTZ | LTO | CARRY  |
|     | RTO | 6      |
|     | RTO | PIF    |

RAD 2-3.3 REGISTER-REGISTER INSTRUCTIONS. The symbolic format of the register-register instructions is:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | OP        | S,D     |

RAN The label field is optional. S and D are expressions which, when evaluated, denote the source and destination registers, respectively.

Typical examples are:

|     |          |
|-----|----------|
| RDE | 0,0      |
| RMO | L,P      |
| ROR | A,STATUS |

2-3.4 REGISTER SKIP INSTRUCTIONS. The symbolic format of register skip instructions is:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | OP        | REG     |

SEV  
SMI  
SNO  
SNZ  
SOD  
SOO  
SPL  
SZE

The label field is optional. REG is an expression for the number of the register involved.

Typical examples are:

|     |   |
|-----|---|
| SEV | A |
| SPL | M |

2-3.5 STATUS INDICATOR SKIP INSTRUCTIONS. The status indicator skip instruction symbolic format is:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | OP        |         |

SEQ  
SGE  
SGT  
SLE  
SLT  
SNC  
SNE  
SNV  
SOC  
SOV

The label field is optional.

Typical examples are:

|     |
|-----|
| SLE |
| SOC |

2-3.6 SENSE SWITCH SKIP INSTRUCTIONS. The symbolic format of the sense switch skip instructions is:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | OP        | SS      |

SSE  
SSN

The label field is optional. SS is an expression which, when evaluated, specifies the sense switches to be tested. For example, to test sense switches 1, 2, 3, or 4 separately; SS must be decimal 8, 4, 2, or 1, respectively.

Typical examples are:

|     |   |
|-----|---|
| SSN | 1 |
| SSE | 5 |



**LRF** 2-3.7 MULTI-REGISTER INSTRUCTIONS. The symbolic format of multi-register instructions is:

**LSB**  
**LSR**  
**SRF**  
**SSB**

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | @OP       | ADDR    |

The label field is optional. ADDR is an expression which forms the second word of the two word multi-register instruction.

NOTE

ADDR is a full 16-bit address; whereas, the DISP in register-memory instructions is an 8-bit displacement for the program counter or the base register.

Typical examples are:

|      |        |                        |                              |
|------|--------|------------------------|------------------------------|
| SRF  |        | } TWO-WORD INSTRUCTION |                              |
| DATA | REGFIL |                        | USING DATA STATEMENT         |
| @SSB | Y      |                        | TWO-WORD INSTRUCTION USING @ |

**CLC** 2-3.8 BYTE MANIPULATION INSTRUCTIONS. The symbolic format of byte manipulation **MVC** instruction is:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | OP        |         |

The label field is optional.

Typical examples are:

CLC  
MVC

**2-3.9 MEMORY BIT MANIPULATION INSTRUCTIONS.** The symbolic format of memory bit manipulation instructions is:

|      |           |         |
|------|-----------|---------|
| NAME | OPERATION | OPERAND |
|------|-----------|---------|

|       |    |           |
|-------|----|-----------|
| LABEL | OP | BIT, ADDR |
|-------|----|-----------|

SMB0  
SMBZ  
TMB0  
TMBZ

The label field is optional. BIT and ADDR are expressions which when evaluated specify the bit within memory which is to be set or tested. ADDR is an optional, 16-bit address. The use of BIT and ADDR is as follows. The expression BIT is evaluated and divided by 16. The quotient is added to the value of the expression ADDR, and this sum forms the second word of this two-word instruction. It identifies the word containing the bit to be set or tested. The remainder of the division of BIT by 16 specifies the particular bit to be set or tested.

For example, the following produce exactly the same code:

|           |         |
|-----------|---------|
| OPERATION | OPERAND |
|-----------|---------|

|      |          |
|------|----------|
| SMBZ | 25, ABC  |
| SMBZ | 9, ABC+1 |

The assembler directive FLAG may be used in conjunction with memory bit manipulation instructions to allow omission of the address field. FLAG supplies a value for ADDR to be used in the memory bit manipulation instructions that follow, provided no address is specified. For example, the following is equivalent to the previous examples:

|           |         |
|-----------|---------|
| OPERATION | OPERAND |
|-----------|---------|

|      |     |
|------|-----|
| FLAG | ABC |
| SMBZ | 25  |

**2-3.10 REGISTER BIT MANIPULATION INSTRUCTIONS.** The symbolic format of register bit manipulation instructions is:

|      |           |         |
|------|-----------|---------|
| NAME | OPERATION | OPERAND |
|------|-----------|---------|

|       |    |     |
|-------|----|-----|
| LABEL | OP | BIT |
|-------|----|-----|

SAB0  
SABZ  
TAB0  
TABZ

The label field is optional. BIT is an expression which when evaluated specifies the bit which is to be set or tested.

Typical examples are:

|      |        |
|------|--------|
| TAB0 | 2      |
| SABZ | OVRFLO |

**API 2-3.11 API INSTRUCTION.** The symbolic format of the API instruction is:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | API       | DEV     |
| LABEL | DATA      | ADDR    |

The label field is optional. DEV is an expression which, when evaluated, indentifies both the auxiliary processor and the command that extends the instruction set.

Although the assembler does recognize the symbolic notation above, most users will find it more convenient to define an extension to the instruction set mnemonics through use of the OPD directive. These extensions should correspond to the instruction set extension provided by the auxiliary processor being used.

For example, if an auxiliary processor for performing vector arithmetic is being used, the following code might be used to perform a dot product.

| NAME | OPERATION | OPERAND | COMMENTS     |
|------|-----------|---------|--------------|
| VLD  | OPD       | >DD00,1 | VECTOR LOAD  |
| VDOT | OPD       | >DD80,1 | VECTOR DOT   |
| SST  | OPD       | >DDC0,1 | SCALAR STORE |
| .    |           |         |              |
| .    |           |         |              |
| .    |           |         |              |
|      | @VLD      | VECT1   |              |
|      | @VDOT     | VECT2   |              |
|      | @SST      | RESULT  |              |
| .    |           |         |              |
| .    |           |         |              |
| .    |           |         |              |

**NOTE**

A vector dot product is defined to be the sum of the product of corresponding vector points.

**RDS 2-3.12 DATA BUS INPUT/OUTPUT INSTRUCTIONS.** The symbolic format is:

**WDS**

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | OP        | DEV     |
| LABEL | DATA      | BIDAR   |

The label fields are optional. DEV is an expression the value of which specifies register and group. The value is treated as a 16-bit number. It is tested for ones in bits 0-4, 7, 8 and 10. If there are none, it is logically ORed with the OP code.

BIDAR is an expression which is evaluated to define the B, I, D, A, and R bits.

Data bus I/O is covered in detail in the manual, Assembly Language Input/Output for the Model 980A Computer. A typical example is:

```
WDS      >10
DATA     >80
```

2-3.13 DMAC INPUT/OUTPUT INSTRUCTIONS. The mnemonic code provided for the DMAC AT input/output has the symbolic format:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | ATI       | DEV     |
| LABEL | DATA      | ADDR    |

The labels are optional. The value of the DEV expression is used as the least significant eight bits of the ATI. ADDR is the 16-bit address of the first list word.

DMAC I/O is covered in detail in the Assembly Language Input/Output manual. A typical example is:

```
ATI      DEVICE
DATA     LIST
```

2-3.14 SINGLE FUNCTION INSTRUCTIONS. The symbolic format of single function instructions is: IDL NRM

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| LABEL | OP        |         |

The label field is optional. Typical examples are:

```
IDL      1
IDL      0
```

In the first example, the 8 least significant bits are used as a flag to indicate the reason for the idle when the instruction register is displayed. If a flag is not desired in the operand field, a zero must be coded. The assembler does not accept a blank operand field for the IDL instruction.

## 2-4 ASSEMBLER DIRECTIVES.

Assembler directives have formats similar to symbolic instructions but do not directly cause code generation. Instead they are directives to the assembler itself and are provided for storage allocation, program identification, format control, and other similar functions. If labels are used with assembler directives, they are assigned the current location counter value unless otherwise specified. Assembler directives are listed in Appendix A. The directives are covered in alphabetical order as an aid in referencing.

**2-4.1 BES BLOCK ENDING SYMBOL.** This evaluates the operand field and advances the location counter by that amount. If a label is present, it is assigned to the new value of the location counter. BES is similar to BSS, except the label is applied to the first location past the reserved area.

**2-4.2 BRR-BASE REGISTER RESET.** BRR informs the assembler that the base register is not available to the assembler for addressing purposes. The programmer can still specify base register addressing in the mode field. The BRR directive informs the assembler to use the base register for addressing purposes only in the event the mode field specifies that type of addressing. (This is the initial condition of assembly.) Under BRR directive control, if D is the unsigned displacement in register-memory instructions, then  $0 \leq D \leq 255$  when the mode field contains B=1 or else a field size error occurs.

**2-4.3 BRS-BASE REGISTER SET.** The BRS directive informs the assembler of the value the base register will contain at run time. The operand field of the BRS directive defines a 16-bit value which will be placed in the B register by the programmer. When the BRS is used and the assembler encounters subsequent register-memory format instructions that would produce field size errors if program counter relative, the assembler will attempt to generate these base register relative. In this case, if D is an unsigned 16-bit evaluation of the displacement expression and B is the value assumed in the base register, then  $0 \leq D - B \leq 255$  or else a field size error occurs.

An example of BRS usage follows:

| NAME | OPERATION   | OPERAND     | COMMENTS                                            |
|------|-------------|-------------|-----------------------------------------------------|
|      | BRS         | CAT         | DEFINE BASE VALUE TO ASSEMBLER                      |
|      | .           |             |                                                     |
|      | .           |             |                                                     |
|      | .           |             |                                                     |
|      | @LDA<br>RMO | =CAT<br>A,B | PUT ADDRESS OF CAT IN BASE<br>REGISTER              |
|      | .           |             |                                                     |
|      | .           |             |                                                     |
|      | .           |             |                                                     |
|      | STA         | CAT+3       | ASSEMBLER USES BASE RELATIVE<br>ADDRESSING          |
|      | .           |             |                                                     |
|      | .           |             |                                                     |
|      | .           |             |                                                     |
| CAT  | BES<br>BSS  | 350<br>10   | CAT IS DEFINED OUT OF<br>PROGRAM COUNTER REL. RANGE |



2.4.4 **BSS-BLOCK STARTING SYMBOL.** This directive reserves an area of memory. The first location in the reserved area is associated with the label given on the name field of the BSS directive. The location of the area reserved is that defined by the location counter, which is then advanced past the reserved area. Note that no object code is generated by the BSS directive. If the programmer desires some value(s) to be assembled in the reserved area, he must do so by other means.

An example of the BSS directive is given below:

| NAME | OPERATION | OPERAND | COMMENTS                       |
|------|-----------|---------|--------------------------------|
|      | BRU       | TOM     |                                |
| AREA | BSS       | 40      | BRANCH AROUND AREA             |
| TOM  | LDA       | AREA    | RESERVE AREA<br>REFERENCE AREA |

Note that any symbol used as the operand of a BSS must be previously defined.

A common usage of symbols in a BSS operand is an expression which defines the length of a reserved area. In the following example, if the length of TABA is likely to change, but TABB must always be the same length as TABA, it may be symbolically stated.

| NAME | OPERATION | OPERAND   | COMMENTS            |
|------|-----------|-----------|---------------------|
| TABA | BSS       | 50        | MIGHT CHANGE        |
| TABB | BSS       | TABB-TABA | ALWAYS SAME AS TABA |



**2-4.5 BYTE GENERATE BYTE ADDRESS.** When using the byte string manipulation instructions, MVC and CLC, it is necessary to address data using byte rather than word addresses. The BYTE directive may be used to generate these byte addresses. Its usage is similar to that of the DATA directive when generating word addresses.

The symbolic format is:

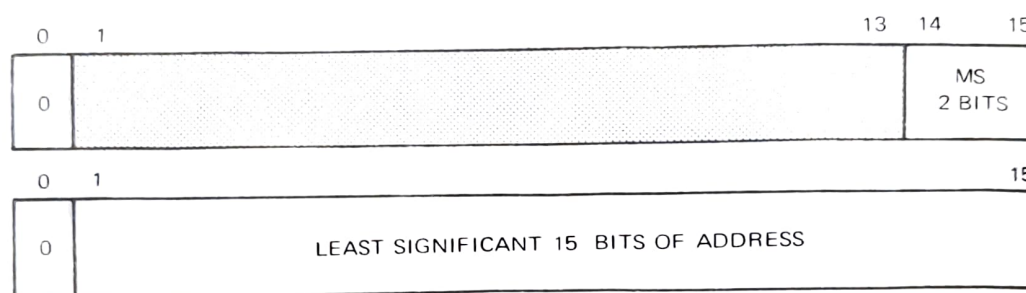
```

NAME      OPERATION      OPERAND
LABEL    BYTE      exp1, exp2, ..., expn

```

LABEL is optional. The operand field contains expressions. These are evaluated and assigned to successive pairs of memory words. If a label is used, it is assigned to the first word of the first byte address.

Byte addresses have the following format:



An expression in a BYTE field is evaluated as a word address and then multiplied by two to obtain the byte address. If the expression is preceded by a colon (:), the byte address is incremented by one. Figure 2-6 illustrates use of the BYTE directive.

|   | LOCATION | CODE | LINE | NAME  | OPERATION | OPERAND                            |
|---|----------|------|------|-------|-----------|------------------------------------|
| P | 0100     | 0107 | 0014 | THERE | DATA      | HERE+2, THERE-6, >100, 100, 0100   |
| P | 0101     | 00FA |      |       |           |                                    |
|   | 0102     | 0100 |      |       |           |                                    |
|   | 0103     | 0064 |      |       |           |                                    |
|   | 0104     | 0040 |      |       |           |                                    |
| P | 0105     | 0105 | 0015 | HERE  | DATA      | HERE, THERE, >100+104, THERE-HERE  |
| P | 0106     | 0100 |      |       |           |                                    |
|   | 0107     | 0168 |      |       |           |                                    |
|   | 0108     | FFFB |      |       |           |                                    |
| P | 0109     | 0000 | 0016 | HERE1 | BYTE      | HERE1                              |
|   | 010A     | 0212 |      |       |           |                                    |
| P | 010B     | 0000 | 0017 |       | BYTE      | :HERE1                             |
|   | 010C     | 0213 |      |       |           |                                    |
| P | 010D     | 0000 | 0018 |       | BYTE      | :HERE+6                            |
|   | 010E     | 0217 |      |       |           |                                    |
| P | 010F     | 0000 | 0019 |       | BYTE      | HERE1, :HERE1, :HERE1+6, >100      |
|   | 0110     | 0212 |      |       |           |                                    |
| P | 0111     | 0000 |      |       |           |                                    |
|   | 0112     | 0213 |      |       |           |                                    |
| P | 0113     | 0000 |      |       |           |                                    |
|   | 0114     | 021F |      |       |           |                                    |
|   | 0115     | 0000 |      |       |           |                                    |
|   | 0116     | 0200 |      |       |           |                                    |
|   | 0117     | 0000 | 0020 |       | BYTE      | >100+>104, THERE-HERE              |
|   | 0118     | 0408 |      |       |           |                                    |
|   | 0119     | FFFF |      |       |           |                                    |
|   | 011A     | FFF6 |      |       |           |                                    |
| P | 011B     | 0000 | 0021 |       | BYTE      | HERE+2, THERE-6, :>100, 100, :0100 |
|   | 011C     | 020E |      |       |           |                                    |
| P | 011D     | 0000 |      |       |           |                                    |
|   | 011E     | 01F4 |      |       |           |                                    |
|   | 011F     | 0000 |      |       |           |                                    |
|   | 0120     | 0201 |      |       |           |                                    |
|   | 0121     | 0000 |      |       |           |                                    |
|   | 0122     | 00C8 |      |       |           |                                    |
|   | 0123     | 0000 |      |       |           |                                    |
|   | 0124     | 0081 |      |       |           |                                    |

FIGURE 2-6. EXAMPLE OF BYTE USAGE

2-4.6 **COMM-BLANK COMMON**. This directive allocates to common storage the number of words indicated by the value of the expression in the operand field. If a label appears, it is assigned a value corresponding to the first word of the block.

Common storage is an area of memory the location of which is not known until load time. The area is *common* because separately assembled programs may share the same common storage. Unlike the **BSS** directive, **COMM** does not reserve area at the current location counter position.

**COMM** is used in a manner similar to **FORTTRAN COMMON**. If a **FORTTRAN** program and assembly language program are merged via link edit, any references in the **FORTTRAN** program to **BLANK COMMON** and references in the assembly language program to **COMM** defined storage are references to the same area of memory. In many applications this simplifies communications between the two programs.

2-4.7 **DATA-GENERATE WORD ADDRESS OR DATA**. A single operation is provided for data generation.

| NAME  | OPERATION | OPERAND                |
|-------|-----------|------------------------|
| LABEL | DATA      | $d_1, d_2, \dots, d_n$ |

LABEL is optional.  $d_1, d_2, \dots, d_n$  are expressions or strings. Expressions are evaluated and assigned to successive memory locations.

The **DATA** statement is used to define alphanumeric strings using the following format:

| NAME  | OPERATION | OPERAND  |
|-------|-----------|----------|
| LABEL | DATA      | 'STRING' |

STRING is a string of characters enclosed in single quotes. The string will be produced in ASCII code, two characters per word, packed left to right. If there is an odd number of characters in the string the last word contains a **blank** code in the last character position.

If a label is used, it is assigned to the first memory location involved. Figure 2-7 gives examples of several types of operands which may be used in a **DATA** statement.

| LOCATION CODE | LINE | NAME | OPERATION | OPERAND | COMMENTS                           |
|---------------|------|------|-----------|---------|------------------------------------|
| 0A03          | 0001 | 0997 | CAT       | DATA    | 1,12,012,>12,'12',CAT,, \$ COMMENT |
| 0A04          | 000C |      |           |         |                                    |
| 0A05          | 000A |      |           |         |                                    |
| 0A06          | 0012 |      |           |         |                                    |
| 0A07          | B1B2 |      |           |         |                                    |
| P 0A08        | 0A03 |      |           |         |                                    |
| 0A09          | 0000 |      |           |         |                                    |
| P 0A0A        | 0A0A |      |           |         |                                    |

FIGURE 2-7. EXAMPLE OF DATA USAGE

2-4.8 **DEF** DEFINE ENTRY POINT SYMBOL. The program-linking assembler directives DEF and REF allow the programmer to symbolically link independently assembled programs which are to be loaded and executed together. Symbolic linkages between programs are created by means of symbols defined in one program and used as operands in another program. Such symbols are termed linkage symbols. A linkage symbol is called a *defined entry point symbol* in the program in which it is defined; it is a *referenced external symbol* in the program in which it is used as an operand.

Every linkage symbol must be properly identified as such in the source program. A linkage symbol used as an external symbol is identified in each using program by the REF directive. A linkage symbol used as an entry point must be identified in the defining program by the DEF directive.

| NAME | OPERATION | OPERAND                               |
|------|-----------|---------------------------------------|
| DEF  |           | $symbol_1, symbol_2, \dots, symbol_n$ |

The relocatable symbols (separated by commas) in the operand field are defined elsewhere in the program and may be used as an entry point by other programs. A symbol that is not defined in the program is flagged in the listing as an error.

In the following sequence, SQRT is identified as an entry-point symbol.

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
| SUBRO | BSS       | 10      |
|       | DEF       | SQRT    |
|       | .         |         |
|       | .         |         |
|       | .         |         |
| SQRT  | STA       | SAVE    |

2-4.9 **END-END OF SOURCE**. The END directive terminates the assembly of a program. It also supplies a point in the program to which control is transferred after the program is loaded.

The END directive must always be the last statement in the source program.

The format of the END directive statement is:

| NAME | OPERATION | OPERAND             |
|------|-----------|---------------------|
|      | END       | <i>exp or blank</i> |

The expression in the operand field specifies the point to which control is transferred when loading is complete. If the operand field is invalid, the statement is flagged as a possible error. If the operand field is blank, the first word of the program is taken as the point for transfer of control.

The point to which control usually is transferred is the first instruction in the program, as shown in this sequence:

| NAME  | OPERATION | OPERAND |
|-------|-----------|---------|
|       | ORG       | >2000   |
| AREA  | BSS       | 50      |
| BEGIN | LDA       | =3      |
|       | .         |         |
|       | .         |         |
|       | .         |         |
|       | END       | BEGIN   |

Here control will be transferred to BEGIN at location 2050. If the operand field were blank, control would be transferred to location 0000 which would be a point outside of this program. When several object programs are joined by link editing, one is specified as the main program. Its transfer point is taken as the transfer point for the link edited program.



2-4.10 **EQU-EQUATE**. The EQU directive is used to define a symbol by assigning to it the value of an expression in the operand field. The format of the EQU statement is:

|        |           |            |
|--------|-----------|------------|
| NAME   | OPERATION | OPERAND    |
| SYMBOL | EQU       | <i>exp</i> |

The symbol in the name field is given the same value as the expression. The expression in the operand field can be relocatable or absolute, and the symbol is similarly defined. Any symbols in the expression must be previously defined.

If the expression in the operand field or the symbol in the name field, or both, are invalid, or are not present, the EQU statement is flagged as an error in the listing and is not used.

The EQU directive is the usual way of equating symbols to register numbers, input/output unit numbers, immediate data, actual addresses, and other arbitrary values. The examples below illustrate how this might be done:

|       |           |         |                   |
|-------|-----------|---------|-------------------|
| NAME  | OPERATION | OPERAND | COMMENTS          |
| REGX  | EQU       | 2       | REGISTER X        |
| IO125 | EQU       | 125     | INPUT/OUTPUT DATA |
| TEST  | EQU       | >3F     | IMMEDIATE DATA    |
| TIMER | EQU       | 80      | ACTUAL ADDRESS    |

To reduce programming time, the programmer can equate symbols to frequently used compound expressions and then use the symbols as operands in place of the expressions. Thus in the statement

|       |           |                  |
|-------|-----------|------------------|
| NAME  | OPERATION | OPERAND          |
| FIELD | EQU       | ALPHA-BETA+GAMMA |

FIELD is defined as ALPHA-BETA+GAMMA and may be used in place of it. Note, however, that ALPHA, BETA, and GAMMA must all be previously defined.

2-4.11 **FLAG-FLAG BIT ADDRESS**. This directive will give an address which will be used by the assembler as a relative starting address for memory bit-referencing instructions. The FLAG directive may be used at any time. The relative memory address will be 0000 until the FLAG directive is defined. An example of the FLAG directive is in paragraph 2-3.9.

2-4.12 **FRM-FORMAT A NEW INSTRUCTION**. Assign the label as an operation code. The operand field is of the form  $exp_1, exp_2, \dots, exp_n$  where the expression values are positive and the sum is 16. When the label is used as an OP code,  $n$  fields are evaluated, truncated to the length specified by the corresponding  $exp_n$ , and placed in the output word.

Example:

|      | NAME | OPERATION | OPERAND     |
|------|------|-----------|-------------|
| 0010 | ABC  | FRM       | 5,5,6       |
|      |      | .         |             |
|      |      | .         |             |
|      |      | .         |             |
| 1000 | F846 | 0020      | ABC >1F,1,6 |

In the first line of this example, ABC is defined to have three fields of 5, 5, and 6 bits, respectively. When ABC is subsequently used as an operation code, the assembler puts 1F in the first 5 bits, 1 in the next 5 bits, and 6 in the last 6 bits of the instruction. Thus, the second line in this example shows the assembled instruction 1111 1000 0100 0110<sub>2</sub> or F846<sub>16</sub>.

**2-4.13 HED-PAGE HEADING.** The remaining characters in the line are printed as page headings on the output listing. The first HED is used as the heading of all pages up to and including the page containing the second HED. Subsequent HED directives appear as page headings on the first page following the one on which the HED appears, and subsequent pages, until another HED is encountered.

**2-4.14 IDT-OBJECT IDENTIFIER.** The symbol appearing in the operand field is reproduced in the object program as the program name. Names less than six characters have trailing blanks. If the name has more than six characters, the output will be truncated, and the name will consist of the first six characters. If the IDT directive is not present, the name will consist of six asterisks.

**2-4.15 IF-CONDITIONAL ASSEMBLY.** The assembly process is altered in accordance with the results of a conditional test. The operand field consists of two expressions and an optional symbol. The two expressions are evaluated and compared. If they are not equal, the assembly process continues with the next line. If the values are equal, the assembly process is suspended. If only two expressions are contained in the operand field, assembly is suspended for one line. If three expressions occur, the third is a label following the IF statement, in which case the assembly process is suspended until the input line is detected with this symbol in its name field.

All lines suspended from the assembly process are treated as comments; i.e., they are printed but no code is generated. Two or more IF statements may have overlapping ranges.

This directive allows assembly-time modification of a program

NOTE

Mathematical expressions cannot be used in the third (label) field.

```
TTYVAL EQU 2
```

TEST ASSUMES ONE DATA TERMINAL AT STANDARD ADDRESS

```
ASR EQU 2
TIP EQU 3
```

TTY1 - ASSUMED ASR AT STANDARD ADDRESS  
TTY2 ASSUMED TIP AT STANDARD ADDRESS

```
TYPE1 CRA 3
      IF TTYVAL,ASR
      WDS TIP
      IF TTYVAL,TIP
      WDS ASR
      DATA BIT8ON
      BRU $-2
```

ROTATE 50 CHARS PRINT OK  
IF TTYVAL = 2, REF DATA TERM 2  
IF TTYVAL = 1, REF DATA TERM 1

2-4.16 **LIS-START LISTING**. Print the assembly listing until the UNL directive is encountered. If it is desired to produce a complete listing, no LIS directive is required.

2-4.17 **OPD-OPERATION DEFINE**. The label is assigned as an operation code mnemonic. The first item in the operand field is evaluated as a 16-bit number and stored as the corresponding operation code. The second item in the operand field indicates the format type for the object instruction. When this label appears as an operation code mnemonic, it is assembled in the appropriate format with the assigned operation code and output in the object program. Any defined operation takes precedence over the standard symbolic operation codes. Format type codes are as follows:

|         |                                                      |
|---------|------------------------------------------------------|
| SPACE - | Register-Memory                                      |
| 0 -     | Register-Memory                                      |
| 1 -     | Register-Memory                                      |
| 2 -     | Register-Register                                    |
| 3 -     | Register Shift                                       |
| 4 -     | Register Skip                                        |
| 5 -     | Status Indicator Skip and Idle                       |
| 6 -     | Data Bus Input/Output                                |
| 7 -     | Sense Switch Skip and Register Bit                   |
| 8 -     | Direct Memory Access Channel and Auxiliary Processor |

#### NOTE

OPD should not be used to define the following operations: NRM, CLC, MVC, memory bit instructions, and multi-register instructions.

The final merging of the operation code and the operand fields is performed using a logical *or*. Thus the operation code may be used to force setting of any bit to one. For example:

|                |      | NAME | OPERATION | OPERAND | COMMENTS      |
|----------------|------|------|-----------|---------|---------------|
|                | 1009 | XYZ  | OPD       | >9800,1 | FORMAT TYPE 1 |
| 0A0C 9AFF 1010 |      | JOE  | XYZ       | JOE,2   | COMMENT       |

In the first line, XYZ is defined to be the mnemonic of an operation code.

The first part of the operand specifies the machine operation code ( $9800_{16}$  or  $1001\ 1000\ 0000\ 0000_2$ ) and the second part of the operand specifies format type 1 or a register-memory format.

In this example, a 5-bit operation code is supplied ( $1001\ 1_2$ ) which results in a hardware multiplication instruction ( $>9800 = \text{MPY}$ ).

Line two shows the assembled result when the defined operation is subsequently used. Format type 1 causes the assembler to look for an optional label, a required operation code, a required first operand field, and an optional second operand field. The operation code (9800) is ored with the IXB tag (2) to produce  $1001\ 1010_2$  or  $9A_{16}$ . The B bit is not set; therefore, the operand is program counter relative. Since the program counter is pointing to the instruction in location 0A0D, the program counter relative address of JOE ( $0A0D - 0001 = 0A0C$ ) is minus one or FF. The ored result produces the machine instruction 9AFF.

Similarly, a new multiply instruction may be defined that is always base register relative by setting the B bit in the first field of the OPD operand as follows:

| NAME | OPERATION | OPERAND |
|------|-----------|---------|
| MPB  | OPD       | >9900,1 |

**2.4.18 ORG-ORIGIN.** This sets the value of the location counter to the value of the expression in the operand field. Any symbol in the expression must be previously defined. If the operand field is invalid, the ORG directive is not used. The ORG directive is commonly used to force loading of a program in specified memory locations, but it may be used to perform other operations. For example:

| OPERATION | OPERAND |
|-----------|---------|
| ORG       | \$+500  |

increases the location counter by 500. Therefore, the ORG directive provides an alternate way to reserve storage areas.

NOTE

If the operand field of any ORG contains an absolute value instead of a relocatable expression, an absolute object is output; otherwise, a relocatable object is output.



2.4.19 **PEJ** PAGE EJECT. The remainder of the current listing page is ejected. The assembler begins a new page headed by the current HED. The PEJ itself is printed as the first line on the new page.

2.4.20 **REF** REFERENCED EXTERNAL SYMBOLS. The REF directive identifies a linkage symbol as an external symbol that is referenced in this program. Each such external symbol must be identified in a REF directive. The format of the REF statement is:

| NAME | OPERATION | OPERAND |
|------|-----------|---------|
|------|-----------|---------|

|     |  |                                       |
|-----|--|---------------------------------------|
| REF |  | $symbol_1, symbol_2, \dots, symbol_n$ |
|-----|--|---------------------------------------|

The external symbols (separated by commas) in the operand field must be defined in another program and identified in that program as an entry-point symbol by the DEF directive.

As an example, if MTPLY is an entry point symbol in another program the using program identifies it as an external symbol, thus:

| OPERATION | OPERAND |
|-----------|---------|
|-----------|---------|

|     |       |
|-----|-------|
| REF | MTPLY |
|-----|-------|

The only way an external symbol may be referenced is as a full 16-bit address. The SAP assembler allows an external symbol to be used in an arithmetic calculation. For example, use of MTPLY+2 is allowed.

To link to a program named SINE, the following coding might be used:

| NAME | OPERATION | OPERAND |
|------|-----------|---------|
|------|-----------|---------|

|       |     |      |
|-------|-----|------|
| PROGA | BSS | 2    |
|       | REF | SINE |

.

.

.

|        |      |      |
|--------|------|------|
| ADSINE | @BRL | SINE |
|--------|------|------|

2.4.21 **UNL** STOP LISTING. Suppress assembly listing until the LIS directive is encountered.

## 2.5 OBJECT FORMAT.

The object program output by the assembler is in the form of standard object records used by all system programs in the Basic System. Information from the IDT and ORG directives is used to generate the header data. If the COMM directive is used, one common area is specified named bBLANK, where b represents an ASCII blank; otherwise, the object contains no common symbols. Entry point records and external reference records are constructed as specified in the DEF and REF directives, respectively. The required text records are created by the assembler, and the end record is generated from the END directive. No block data records are output by the assembler. Details of the object records are covered in the manual, Basic System Use and Operation for the Model 980A Computer.

## 2-6 SAP ERROR MESSAGES.

The various versions of the assembler may detect certain syntax errors in the source program. When this occurs, a diagnostic message (or the message number on SAP4) is printed in the listing adjacent to the line in question. These comments (listed below) apply only to those versions of SAP which operate in the Model 980A Computer. The IBM System/360 SAP produces error messages which are slightly different. These messages are given in the manual, IBM 360 Support Software for the Model 980A Computer.

| Message Number | Message  | Meaning (and Corrective Action)                                                                                                         |
|----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1              | FIELD SZ | Address beyond reach (use @ for extended format)                                                                                        |
| 2              | UNDF OP  | Undefined operation code (check list of valid op codes)                                                                                 |
| 3              | LONG SYM | Symbol > 6 characters                                                                                                                   |
| 4              | MDF O/F  | OPD or FRM multiply defined (rename label)                                                                                              |
| 5              | FRM > 16 | FRM fields contain more than 16 bits                                                                                                    |
| 6              | CAD > 10 | Address expression has > 10 elements                                                                                                    |
| 7              | UNDF SYM | Symbol not defined (label probably omitted)                                                                                             |
| 8              | MDF SYM  | Symbol multiply defined (rename labels)                                                                                                 |
| 9              | RELOC    | A relocation error (use only one relocatable label in arithmetic expression, or ORG statement can use only one relocatable label)       |
| 10             | SYM OVFL | Too many symbols have been defined (cut out symbols or divide program)                                                                  |
| 11             | BAD NUM  | Numeric element not valid (properly define item in label or address field)                                                              |
| 12             | IMP R/D  | A REF or DEF symbol has been used improperly (REF symbol defined inside and outside the program, DEF symbol not defined in the program) |
| 13             | X RF USE | A REF symbol has appeared invalidly in an unrelocatable expression                                                                      |
| 14             | IXB ERR  | Address mode error (improper use of IXB field (Table 3-1 in ALMI manual))                                                               |
| 15             | OPD ERR  | No such format number (OPD format numbers 0 to 8 explained in paragraph 2-4.17)                                                         |
| 16             | ADR MODE | Illegal addressing mode (improperly written address)                                                                                    |



# APPENDIX A

## SYMBOLIC FORM OF MACHINE INSTRUCTIONS

### INSTRUCTION TYPE

### SYMBOLIC FORMAT

### EXAMPLE

Register-Memory  
Register Shift  
Register-Register  
Register Skip  
Status Indicator Skip  
Sense Switch Skip  
Multi-Register  
Byte Manipulation  
Memory Bit Manipulation  
Register Bit Manipulation  
Auxiliary Processor Initiate

Data Bus I/O

DMAC I/O

Single Function

LABEL @OP      =\*DISP,IXB  
LABEL OP      COUNT  
LABEL OP      S,D  
LABEL OP      REG  
LABEL OP      SS  
LABEL @OP      ADDR  
LABEL OP      BIT,ADDR  
LABEL OP      BIT  
LABEL API      DEV  
LABEL DATA      ADDR  
LABEL OP      DEV  
LABEL DATA      BIDAR  
LABEL ATI      DEV  
LABEL DATA      ADDR  
LABEL OP

LDA      BUFFER,X  
CRS      STOP-START  
RAD      E,A  
SNZ      E  
SGE  
SSE      1  
@SRF      \$+50  
MVC  
SMBZ      5,STATUS  
TABO      ERROR  
API      LOAD  
DATA      VECT1  
RDS      >1F  
DATA      >2080  
ATI      5  
DATA      LIST  
IDL

### SYMBOL

### MEANING

\$      This location (current value of the location counter)  
>      Hexadecimal number  
@      Extended format (optional)  
\*      Indirect addressing (optional)  
=      Immediate operand (optional)  
ADDR      Memory location  
BIDAR      Busy, increment, decrement, address bits, and internal register  
D      Destination register  
DEV      External group and device  
DISP      Displacement (in words) from program counter or specified base  
IXB      Indirect, index, and base bits (optional)  
OP      Operation code  
REG      Register number  
S      Source register  
SS      Sense switch

## DATA GENERATION

| NAME | OPERATION | OPERAND               | COMMENTS |
|------|-----------|-----------------------|----------|
| OUT  | DATA      | 10,012,>A,'TEN',LABEL |          |

## ASSEMBLER DIRECTIVES

|        |                               |
|--------|-------------------------------|
| BES    | Block Ending Symbol           |
| * BRR  | Base Register Reset           |
| * BRS  | Base Register Set             |
| * BSS  | Block Starting Symbol         |
| * BYTE | Generate Byte Address         |
| COMM   | Common Storage Allocation     |
| * DATA | Generate Word Address or Data |
| * DEF  | Define Entry Point            |
| * END  | End of Source                 |
| * EQU  | Equate                        |
| * FLAG | Flag Bit Address              |
| FRM    | Format                        |
| HED    | Heading                       |
| * IDT  | Identification                |
| IF     | If (conditional assembly)     |
| LIS    | List                          |
| * OPD  | Operation Define              |
| * ORG  | Origin                        |
| PEJ    | Page Eject                    |
| * REF  | External Reference            |
| * UNL  | Unlist                        |

\*In the 4K version of the assembler, only the directives marked with an asterisk are available.

# APPENDIX B

## IXB ADDRESSING EXAMPLES

| NAME | OPERATION | OPERAND | COMMENTS |
|------|-----------|---------|----------|
|------|-----------|---------|----------|

|      |      |       |  |
|------|------|-------|--|
|      | .    |       |  |
|      | .    |       |  |
|      | .    |       |  |
| A    | EQU  | 0     |  |
| B    | EQU  | 6     |  |
| TWOK | DATA | >2000 |  |

.

.

.

.

.

.

.

### HEXADECIMAL

|     |      |
|-----|------|
| LDA | TWOK |
| RMO | A,B  |
| LDA | =>10 |
| STA | 0,1  |
| STA | 0,5  |
| LDA | =5   |
| STA | 1,1  |
| STA | 1,5  |
| LDX | =2   |
| STA | 0,3  |

| LOCATION | CONTENTS |
|----------|----------|
| REG. A   | 2000     |
| REG. B   | 2000     |
| REG. A   | 0010     |
| 2000     | 0010     |
| 0010     | 0010     |
| REG. A   | 0005     |
| 2001     | 0005     |
| 0005     | 0005     |
| REG. X   | 0002     |
| 2002     | 0005     |

.

.

.



## USER'S RESPONSE SHEET

Manual Title: Model 980A Computer Assembly Language Coding Conventions (961961-9732)

Date of Manual: 8 July 1974 (Change 1) Date of This Letter: \_\_\_\_\_

User: \_\_\_\_\_ Office/Dept. No.: \_\_\_\_\_

Company: \_\_\_\_\_

Street Address: \_\_\_\_\_

City/State/Zip: \_\_\_\_\_

Please list any discrepancy found in this manual by page, paragraph, figure, or table number in the following space. If there are any other suggestions that you wish to make, feel free to include them. Thank you.

Location  
in Manual

Comment/Suggestion

---

---

---

---

---

---

---

---

---

---

---

---

---

---

NO POSTAGE NECESSARY IF MAILED IN U.S.A.  
FOLD ON TWO LINES (LOCATED ON REVERSE SIDE), STAPLE AND MAIL



First Class  
PERMIT NO. 3135  
Austin, Texas

**BUSINESS REPLY MAIL**

No Postage Necessary if Mailed in the United States

Postage Will Be Paid by

**TEXAS INSTRUMENTS INCORPORATED**  
DIGITAL SYSTEMS DIVISION

**P.O. BOX 2909 · AUSTIN, TEXAS 78767**

**Attn: TECHNICAL PUBLICATIONS, MS 2146**





ALCC

Sales and Service Offices of Texas Instruments are located throughout the United States and in major countries overseas. Contact the Digital Systems Division, Texas Instruments Incorporated, P.O. Box 1444, Houston, Texas 77001, or call (713) 494-5115, for the location of the office nearest to you.



Texas Instruments reserves the right to make changes at any time to improve design and supply the best product possible.

**TEXAS INSTRUMENTS**  
INCORPORATED

TI-504